

Logic Design and Switching Theory

Lecture – 6

Subtitle: Functions of Combinational Logic
(Half Adders, Full Adders, Parallel Adders, 1's Complement Adder/Subtractor, and Carry Look-Ahead Adder.)

by
Ms. Hera Noor

Previous

Active High:

- A signal or pin is considered "active high" when its high state (typically represented by a logic '1' or a higher voltage level) corresponds to the active or "ON" condition of a device or function.
- When the signal is high, the associated action or operation is enabled or triggered.
- When the signal is low (logic '0' or lower voltage), the action is inactive or "OFF".

Active Low:

- A signal or pin is "active low" when its low state (logic '0' or lower voltage) signifies the active or "ON" condition.
- In this case, a low signal triggers the associated action or operation.
- A high signal (logic '1' or higher voltage) indicates an inactive or "OFF" state.

Previous

NAND Implementation:

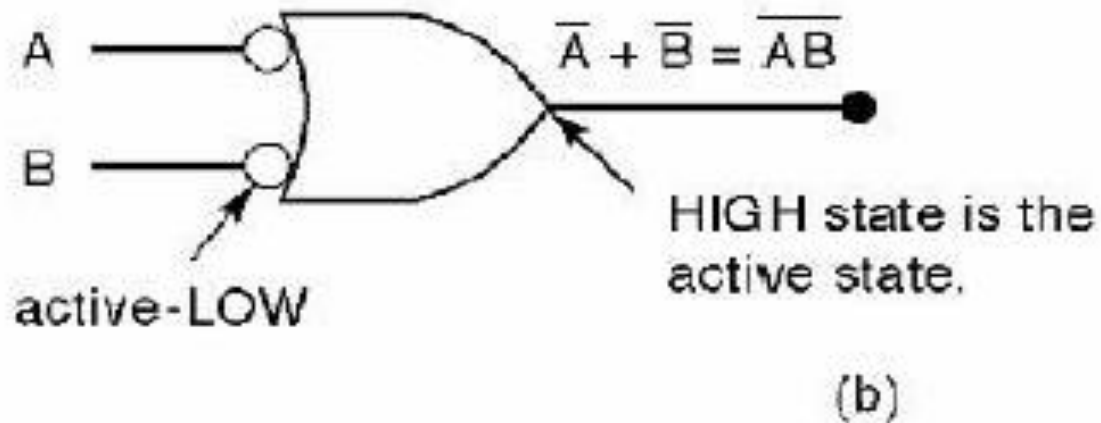
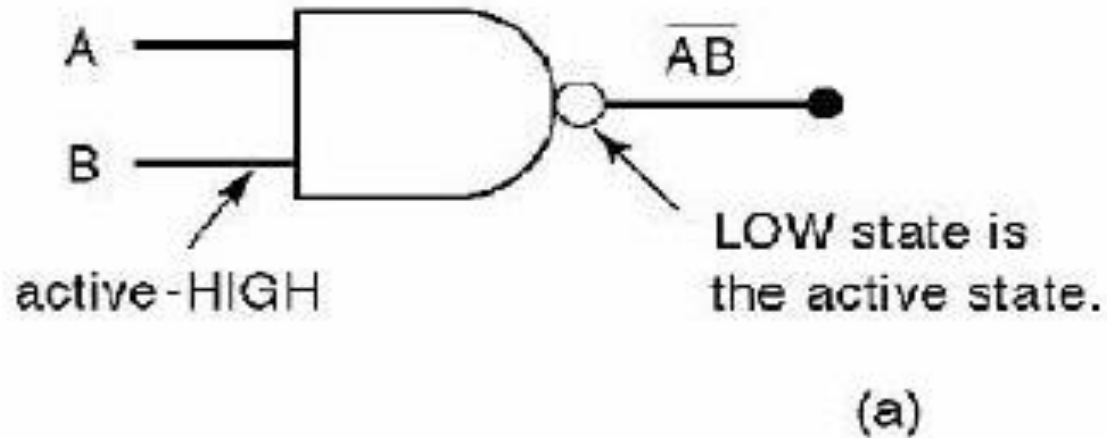
NAND gates are universal gates, meaning they can be used to implement any other logic function. To implement other logic functions using NAND gates, you need to use a combination of NAND gates to achieve the desired output. This involves using NAND gates to create inverters (NOT gates), AND gates, and OR gates.

NOR Implementation:

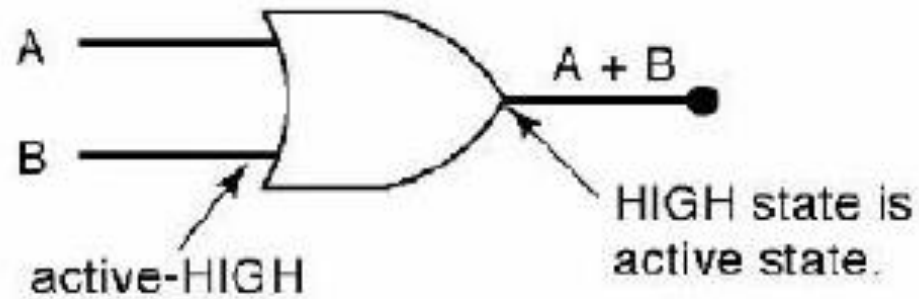
Similar to NAND gates, NOR gates are also universal gates.

- You can implement other logic functions using a combination of NOR gates.

Interpretation of two NAND symbols

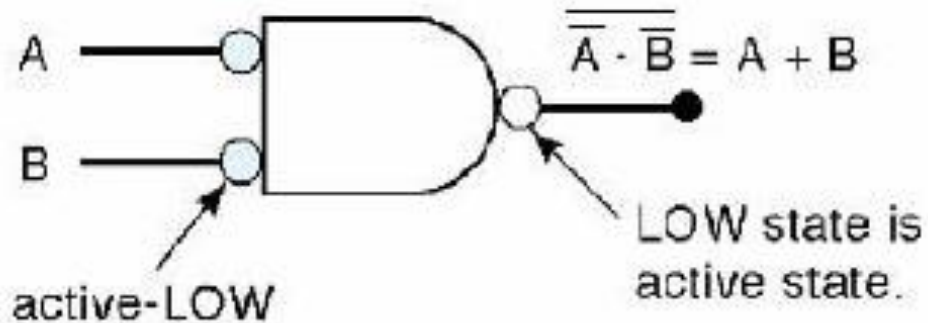


Interpretation of two OR symbols



Output goes HIGH when any input is HIGH.

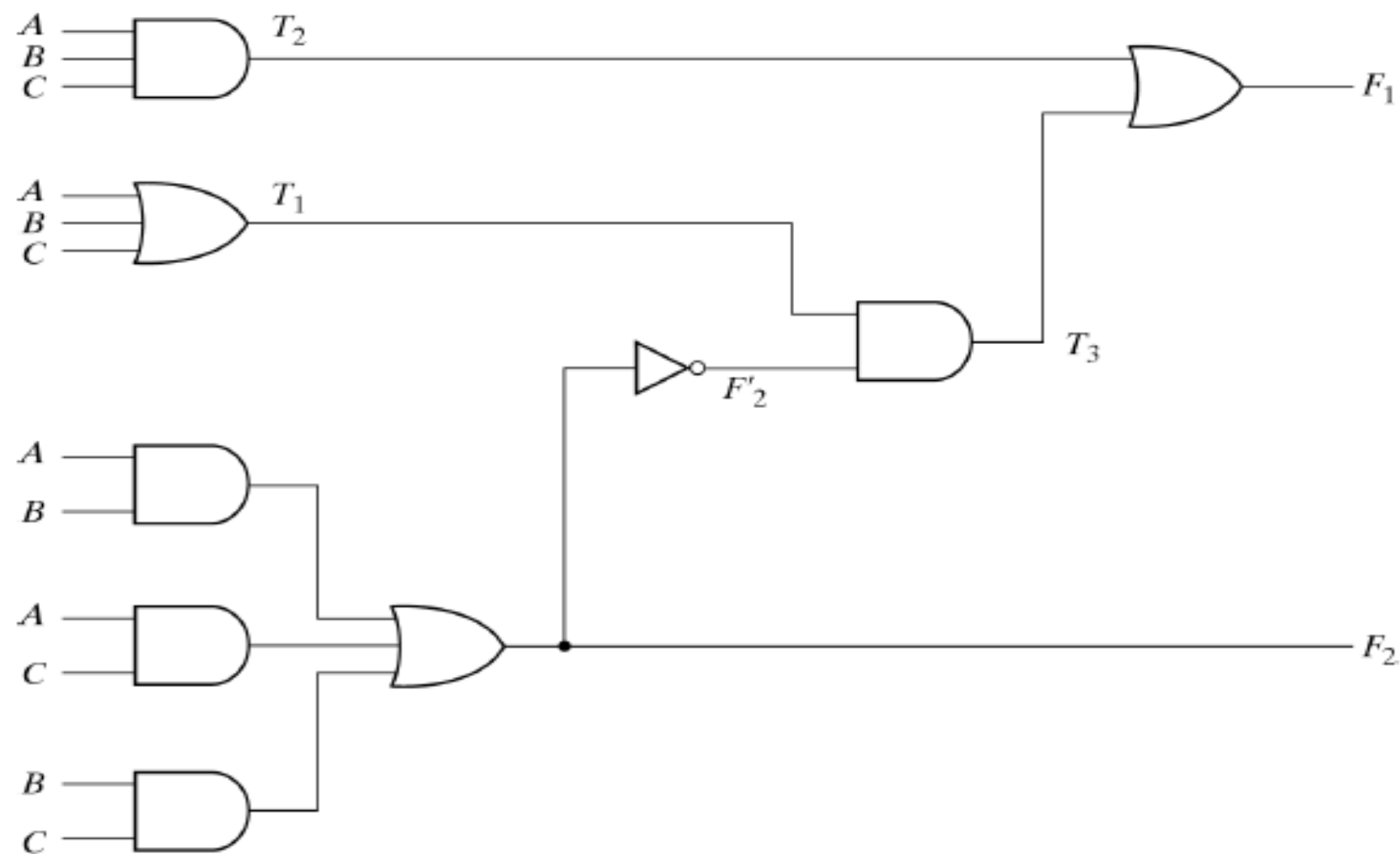
(a)



Output goes LOW only when all inputs are LOW.

(b)

Label all gate outputs that are functions of input variables



Functions of Combinational Logic

(Half Adders, Full Adders, Parallel Adders, 1's Complement Adder/Subtractor, and Carry Look-Ahead Adder.)

BASIC ADDERS

A half-adder adds two bits and produces a sum and a carry output. Adders are important in computers and also in other types of digital systems in which numerical data are processed.

The Half-Adder

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

The operations are performed by a logic circuit called a half-adder. The half-adder accepts two binary digits on its inputs and produces two binary digits on its outputs, a sum bit and a carry bit.

BASIC ADDERS

The Half-Adder

Half-Adder Logic: From the operation of the half-adder as stated in Table 7-1, expressions can be derived for the sum and the output carry as functions of the inputs. Notice that the output Carry (C_{out}) is a 1 only when both A and B are 1s: therefore. C_{out} can be expressed

$$C_{out} = AB$$

Now observe that the sum output (Σ) is a 1 only if the input variables A and B, are not equal. The sum can therefore be expressed as the exclusive-OR of the input variables.

$$\Sigma = A \oplus B$$

BASIC ADDERS

The Half-Adder

Binary Addition: Half adders are designed to add two binary digits (0 or 1).

Outputs: They produce two outputs:

- **Sum:** The sum of the two input bits.
- **Carry:** A carry bit that indicates if the addition result exceeds 1 (e.g., $1 + 1 = 10$ in binary).

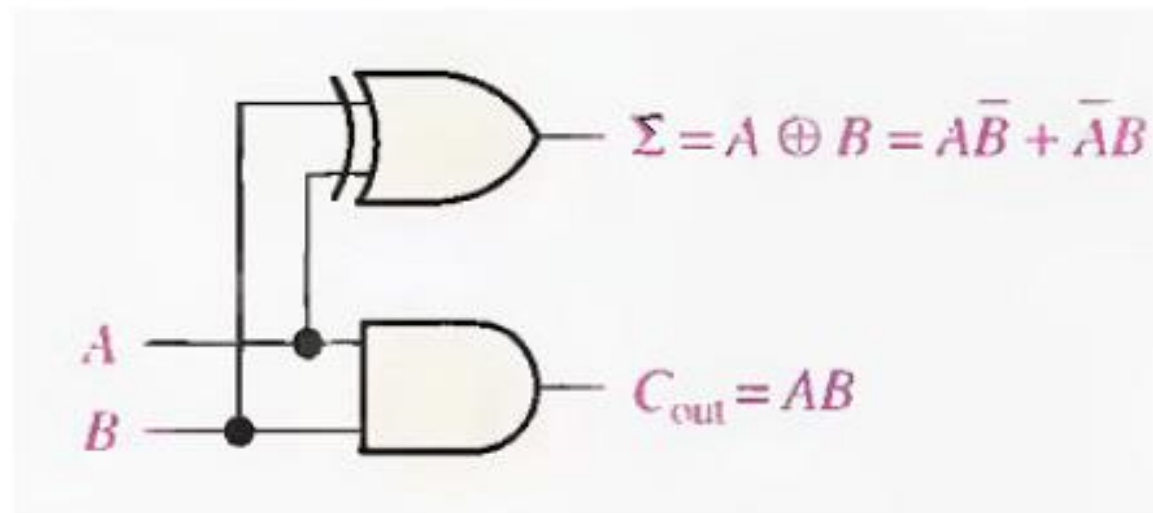
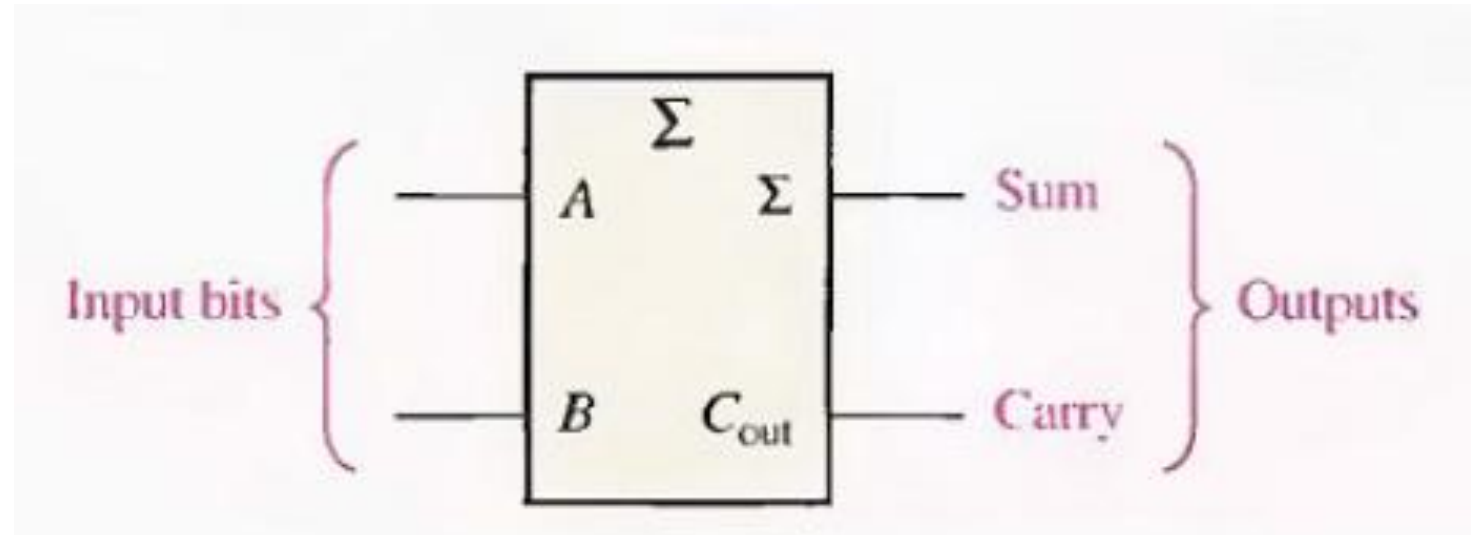
Building Block: Half adders are essential for constructing more complex adders, like full adders, which can handle multi-bit binary addition.

Limitations:

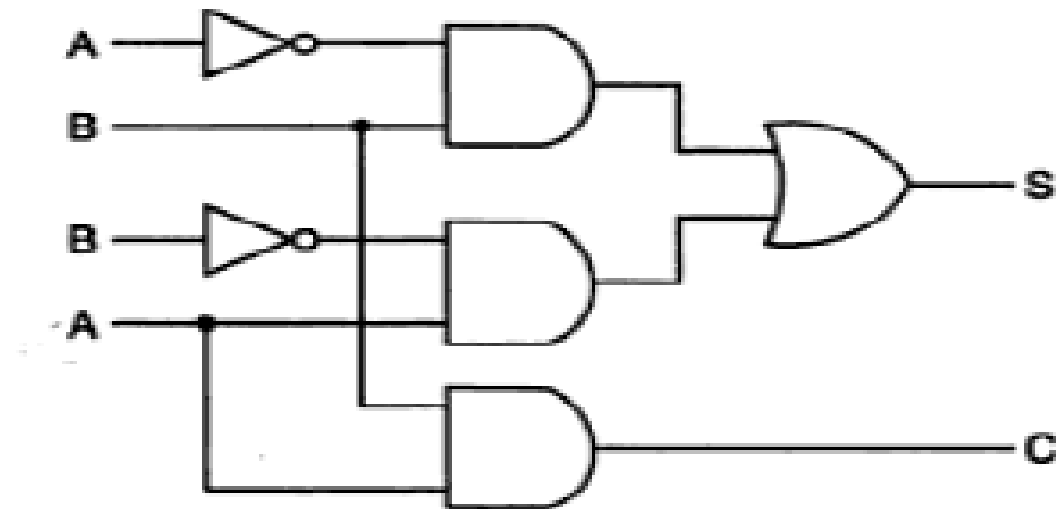
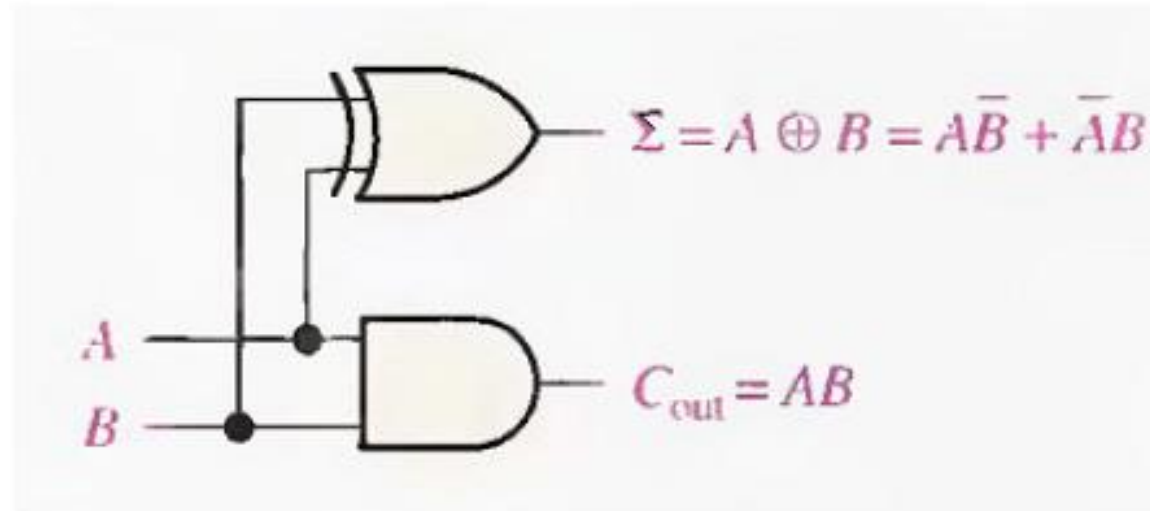
Half adders can only add two bits at a time and do not consider any carry from previous additions. This limitation makes them unsuitable for multi-bit addition on their own.

BASIC ADDERS

The Half-Adder

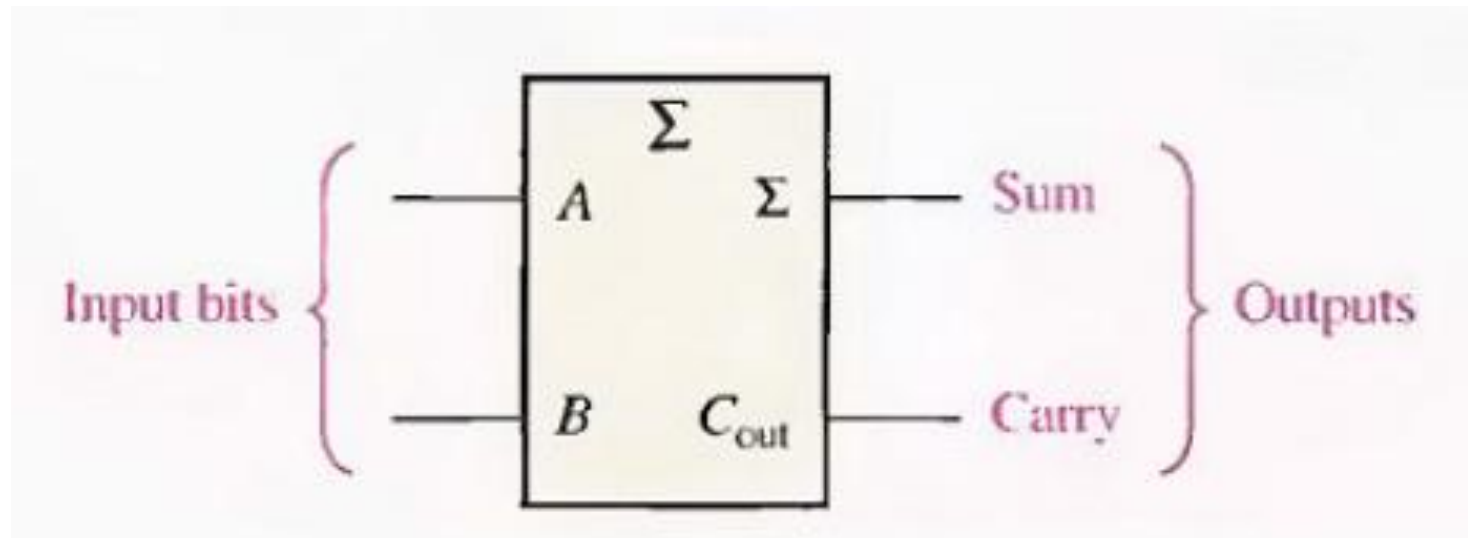


HALF ADDERS



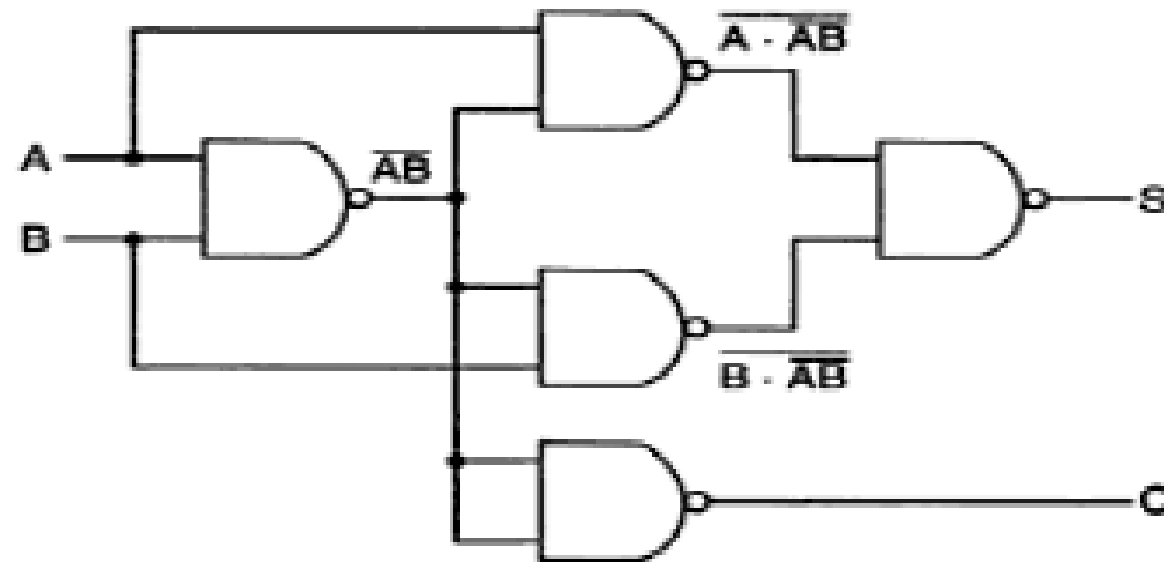
HALF ADDERS

The Half-Adder



Half Adder

$$\begin{aligned} S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \\ &= A \cdot \overline{AB} + B \cdot \overline{AB} \\ &= \overline{A \cdot AB \cdot B \cdot AB} \\ C &= AB = \overline{\overline{AB}} \end{aligned}$$



Logic diagram of a half-adder using only 2-input NAND gates.

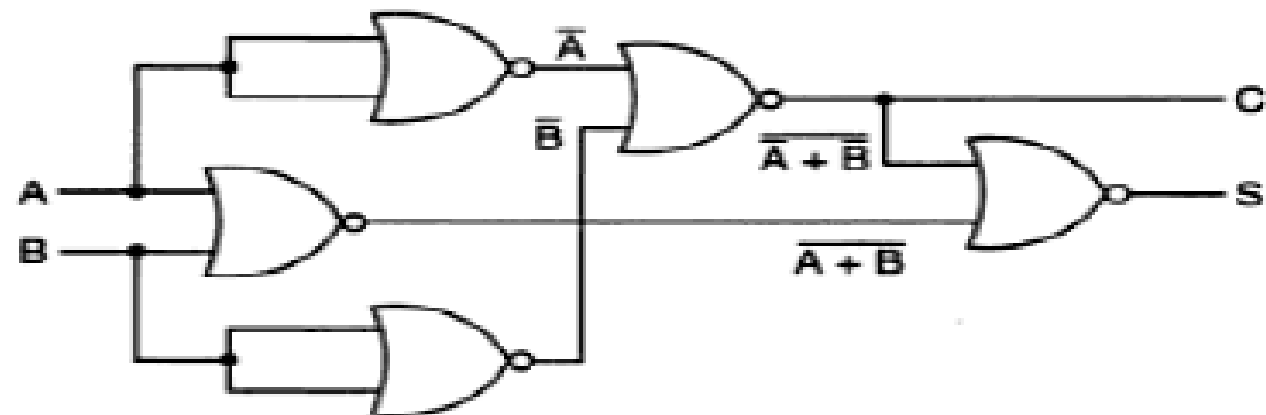
Half Adder

$$\begin{aligned} S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \end{aligned}$$

$$= (A + B)(\bar{A} + \bar{B})$$

$$= \overline{\overline{A + B} + \overline{\bar{A} + \bar{B}}}$$

$$C = AB = \overline{\overline{AB}} = \overline{\overline{A} + \overline{B}}$$



Logic diagram of a half-adder using only 2-input NOR gates.

Full Adder

Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM.

A Half Adder consists of only one AND gate and EX-OR gate. A Full Adder consists of one OR gate and two EX-OR and AND gates. There are two inputs in a Half Adder- A and B. There are a total of three inputs in a Full Adder- A, B, and C-IN.

Full Adder

A Full Adder adds three bits: A, B, and Carry-in (Cin).

Outputs: Sum (S) and Carry-out (Cout)

Logic Equations:

$$\text{Sum (S)} = A \oplus B \oplus \text{Cin}$$

$$\text{Carry-out (Cout)} = (A * B) + (\text{Cin} * (A \oplus B))$$

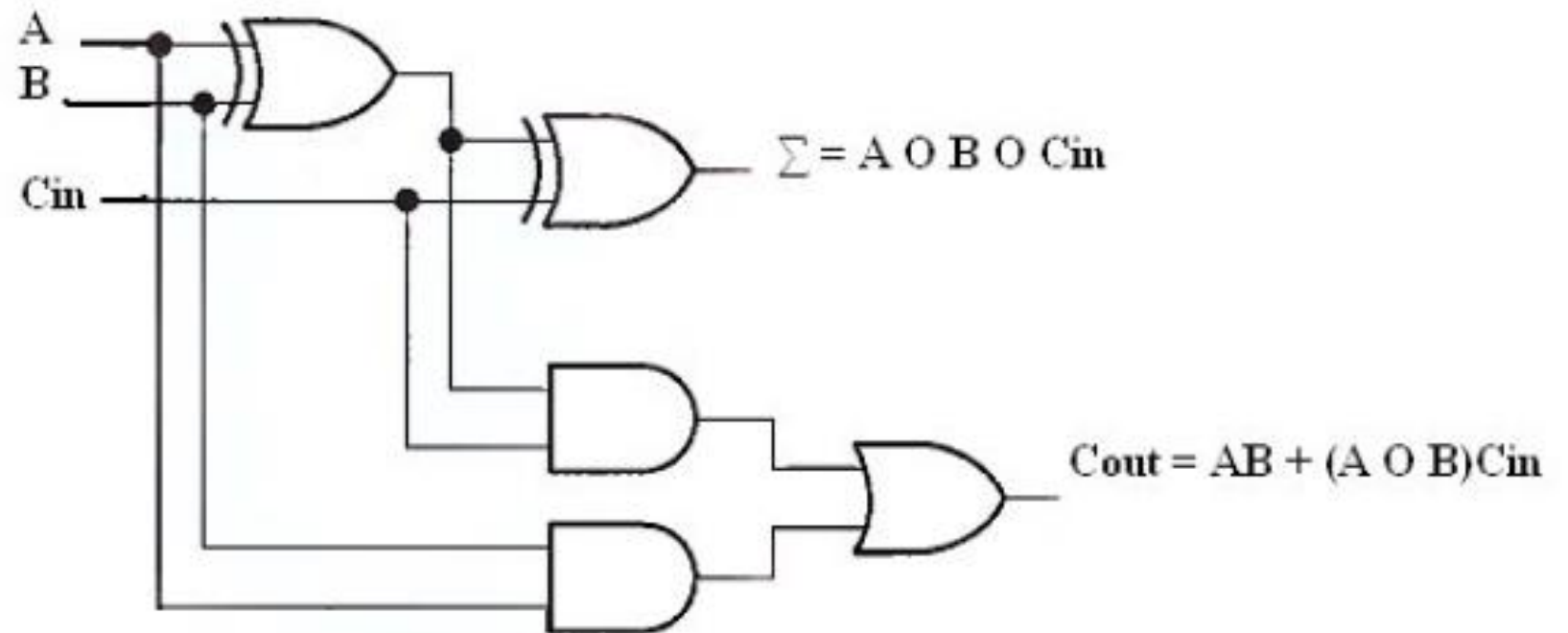
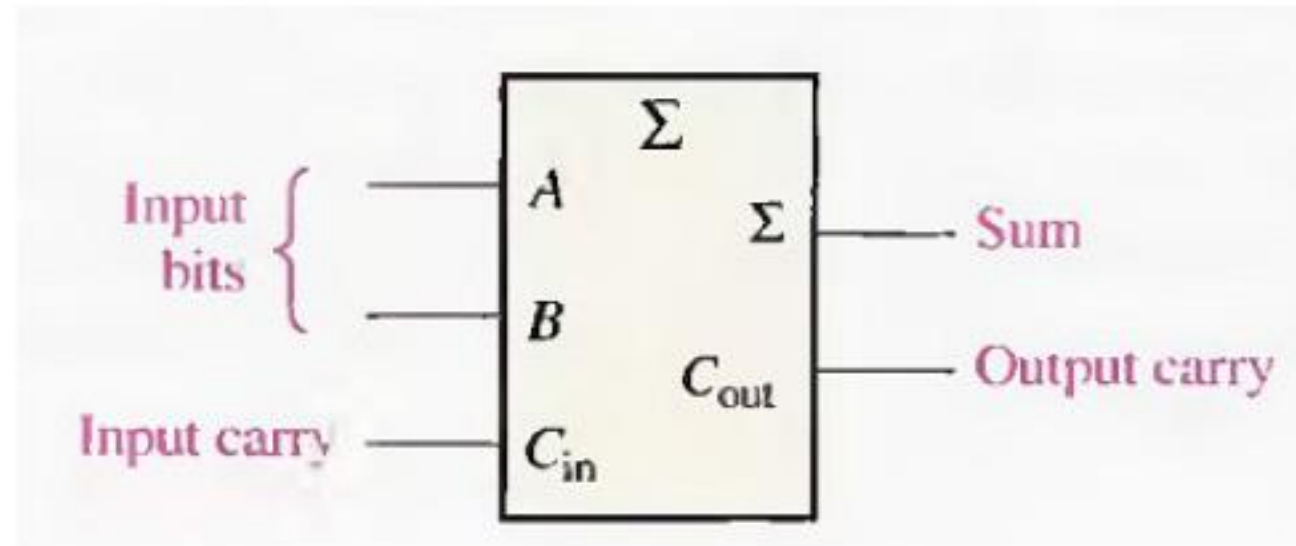
Full Adder

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

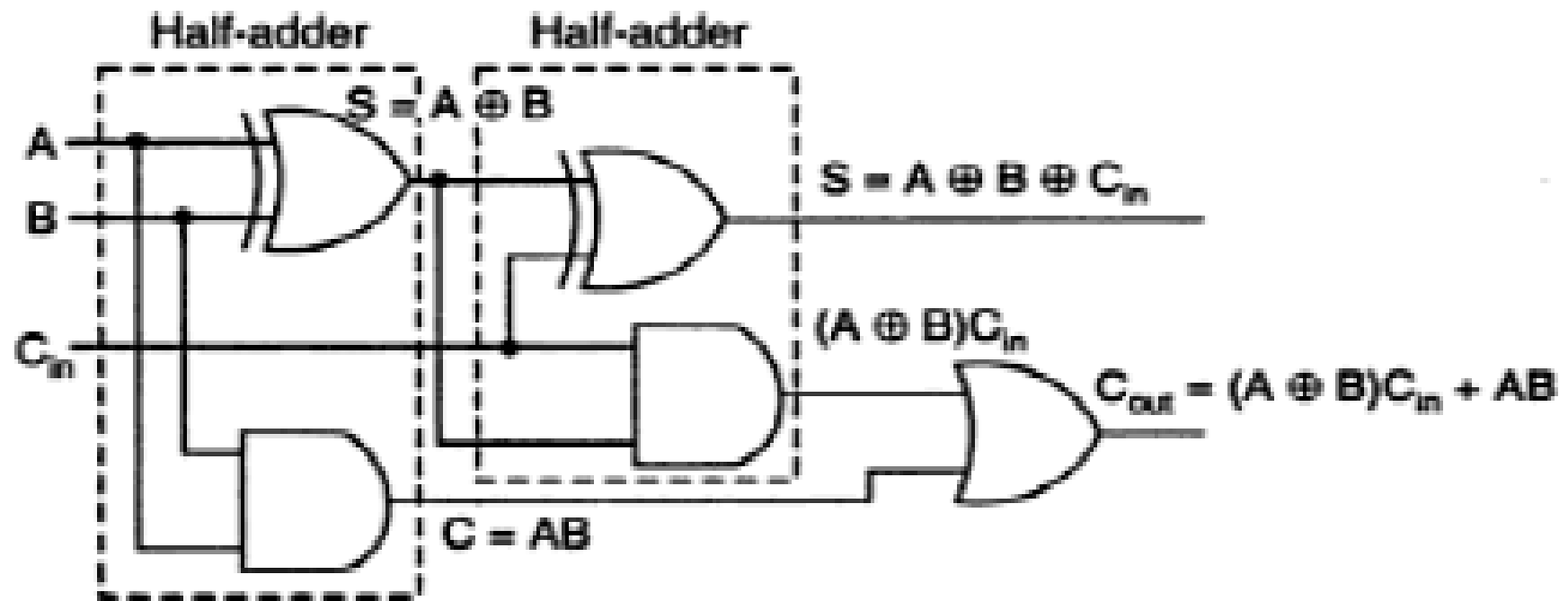
Full Adder

$$\Sigma = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B)C_{in}$$

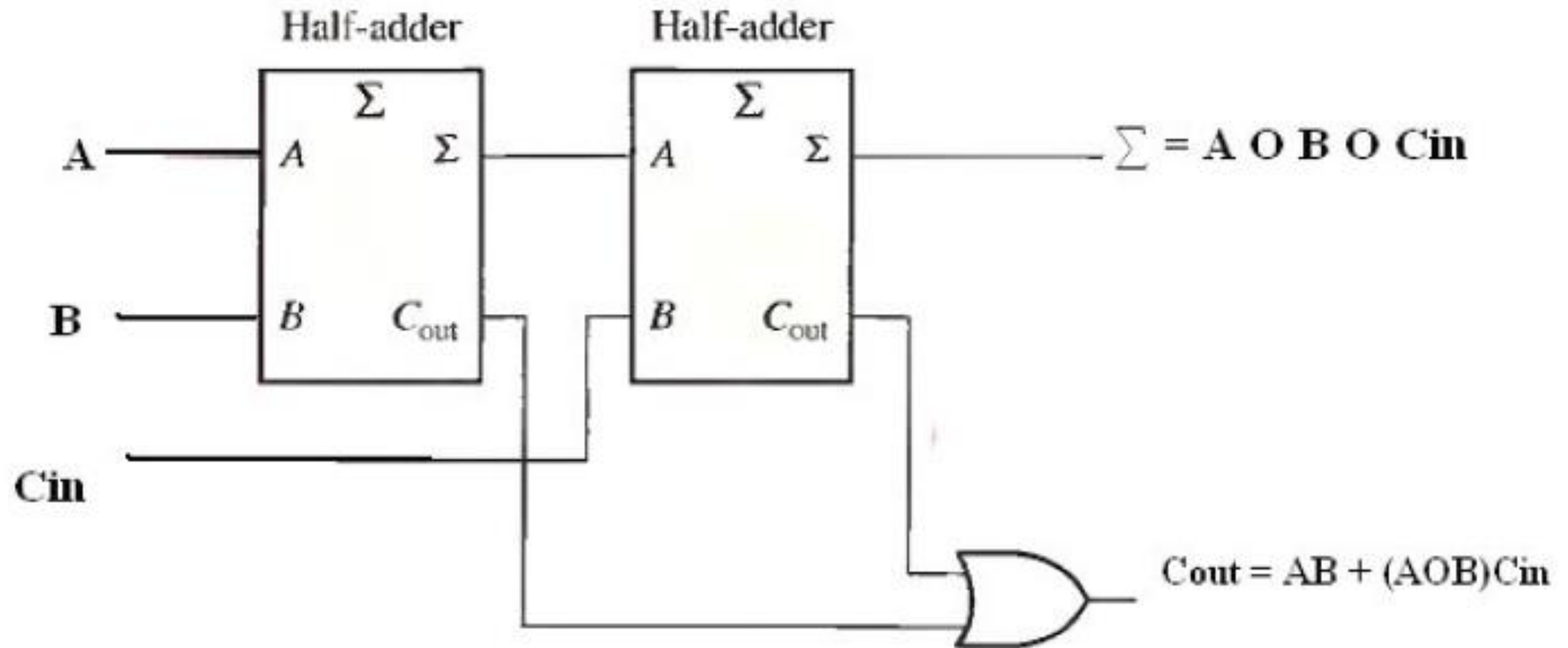


Two half adder to make a Full Adder



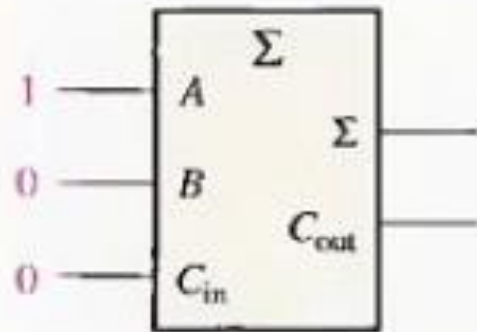
Logic diagram of a full-adder using two half-adders.

Two half adder to make a Full Adder

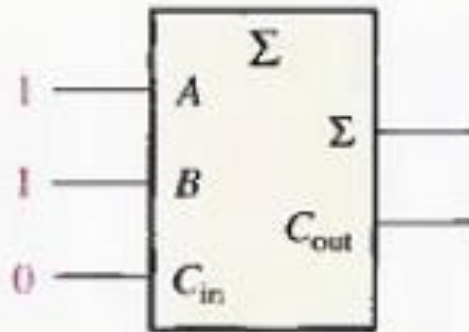


Full Adder

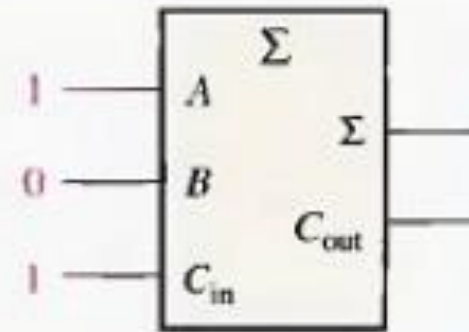
Example: For each of the three full-adders in Fig.(7-6), determine the outputs for the inputs shown.



(a)

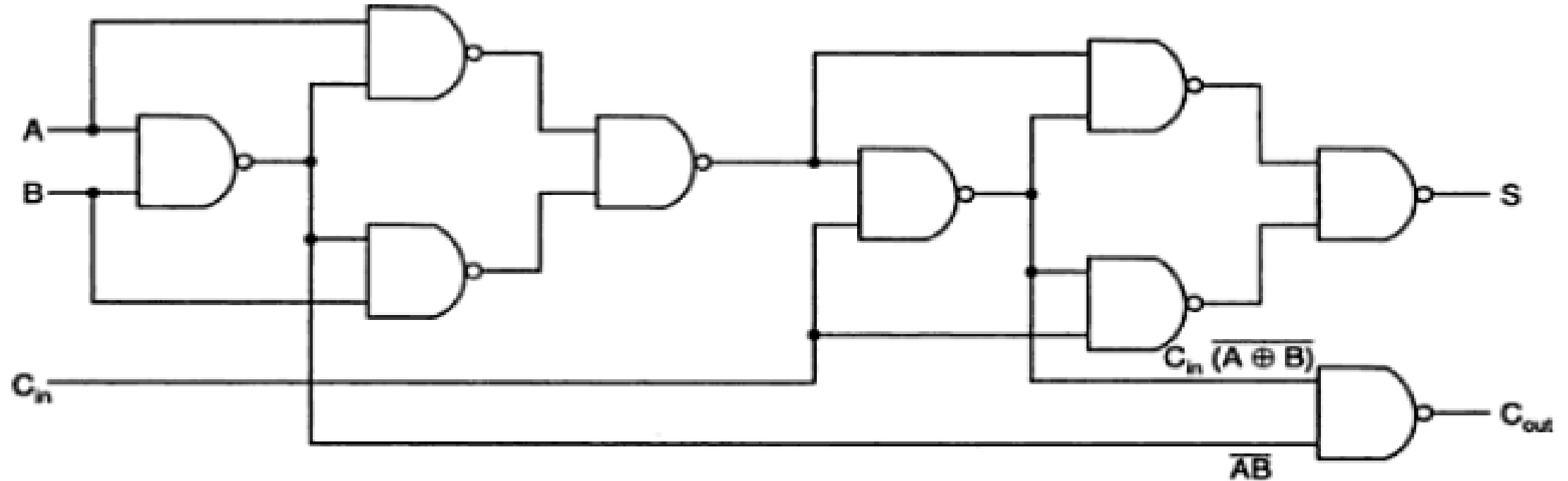


(b)



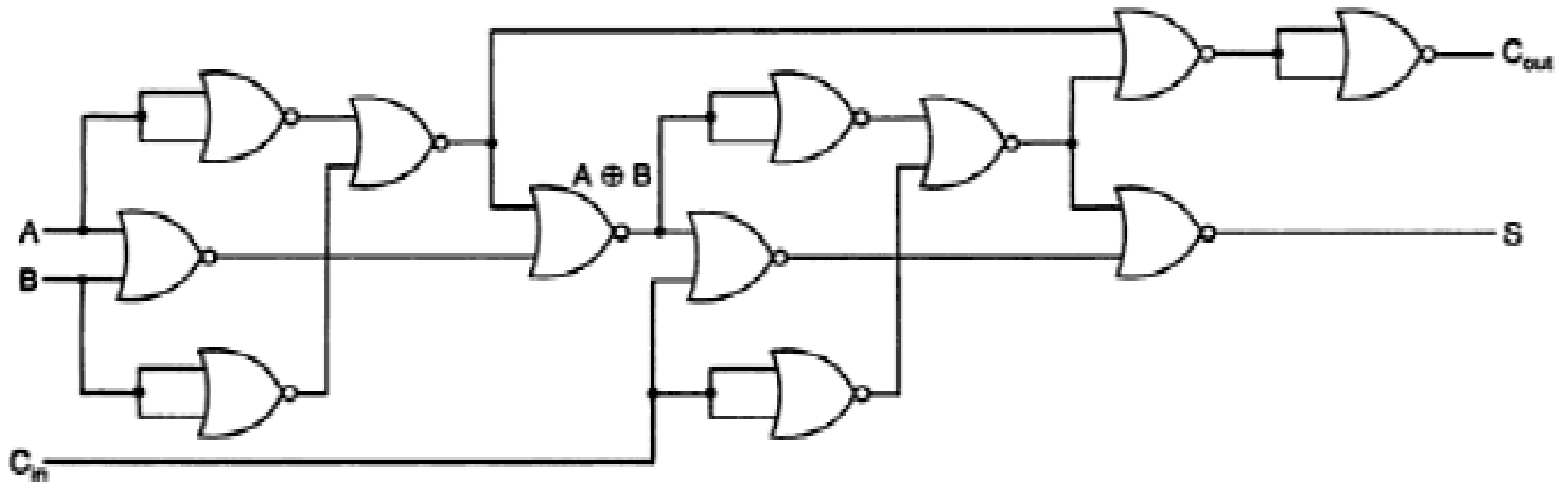
(c)

NAND Logic for Full Adder



Logic diagram of a full-adder using only 2-input NAND gates.

NOR Logic for Full Adder



Logic diagram of a full-adder using only 2-input NOR gates.

Half Subtractor

A Half-subtractor is a combinational circuit with two inputs A and B and two outputs d and b. d indicates the difference and b is the output signal generated that informs the next stage that a 1 has been borrowed. When a bit B is subtracted from another bit A, a difference bit (d) and a borrow bit (b) result according to the rules given as,

Inputs		Outputs	
A	B	d	b
0	0	0	0
1	0	1	0
1	1	0	0
0	1	1	1

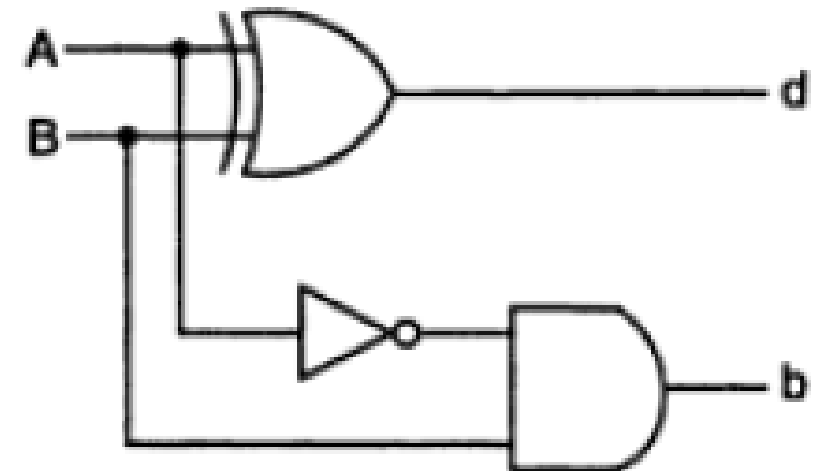
(a) Truth table

Half Subtractor

A Half-subtractor is a combinational circuit with two inputs A and B and two outputs d and b. d indicates the difference and b is the output signal generated that informs the next stage that a 1 has been borrowed. When a bit B is subtracted from another bit A, a difference bit (d) and a borrow bit (b) result according to the rules given as,

Inputs		Outputs	
A	B	d	b
0	0	0	0
1	0	1	0
1	1	0	0
0	1	1	1

(a) Truth table

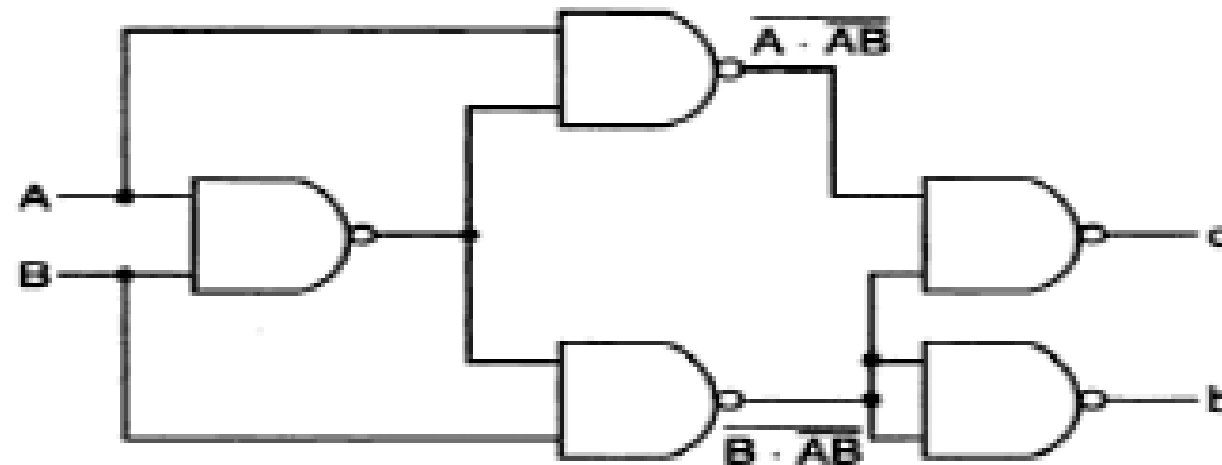


Half Subtractor

NAND Logic:

$$d = A \oplus B = \overline{\overline{A \cdot \overline{AB}} \cdot B \cdot \overline{AB}}$$

$$b = \overline{A}B = B(\overline{A} + \overline{B}) = B(\overline{AB}) = \overline{B \cdot \overline{AB}}$$

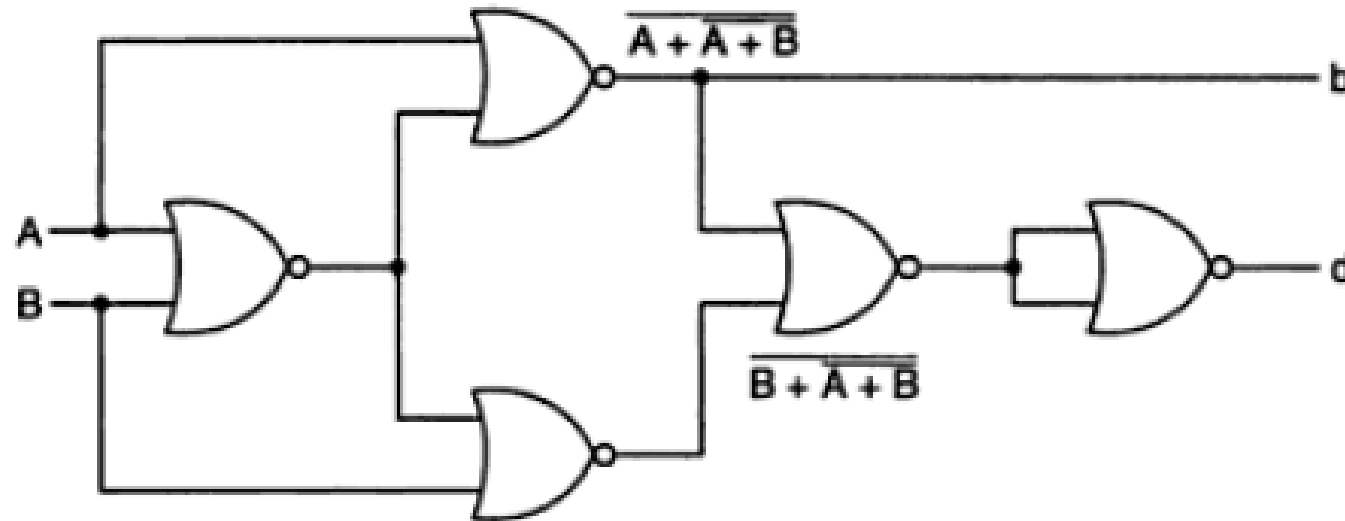


Logic diagram of a half-subtractor using only 2-input NAND gates.

Half Subtractor

NOR Logic:

$$\begin{aligned}d &= A \oplus B = A\bar{B} + \bar{A}B = A\bar{B} + B\bar{B} + \bar{A}B + A\bar{A} \\&= \bar{B}(A + B) + \bar{A}(A + B) = \overline{\overline{B} + \overline{A + B}} + \overline{\overline{A} + \overline{A + B}} \\d &= \bar{A}B = \bar{A}(A + B) = \overline{\overline{\bar{A}(A + B)}} = \overline{A + (\bar{A} + \bar{B})}\end{aligned}$$



Logic diagram of a half-subtractor using only 2-input NOR gates.

Full Subtractor

The half-subtractor can be only for LSB subtraction. IF there is a borrow during the subtraction of the LSBs, it affects the subtraction in the next higher column; the subtrahend bit is subtracted from the minuend bit, considering the borrow from that column used for the subtraction in the preceding column. Such a subtraction is performed by a full-subtractor.

It subtracts one bit (B) from another bit (A) , when already there is a borrow b_i from this column for the subtraction in the preceding column, and outputs the difference bit (d) and the borrow bit(b) required from the next d and b. The two outputs present the difference and output borrow. The 1s and 0s for the output variables are determined from the subtraction of $A-B-b_i$.

Full Subtractor

Inputs			Difference	Borrow
A	B	b_1	d	b
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

(a) Truth table

Full Subtractor

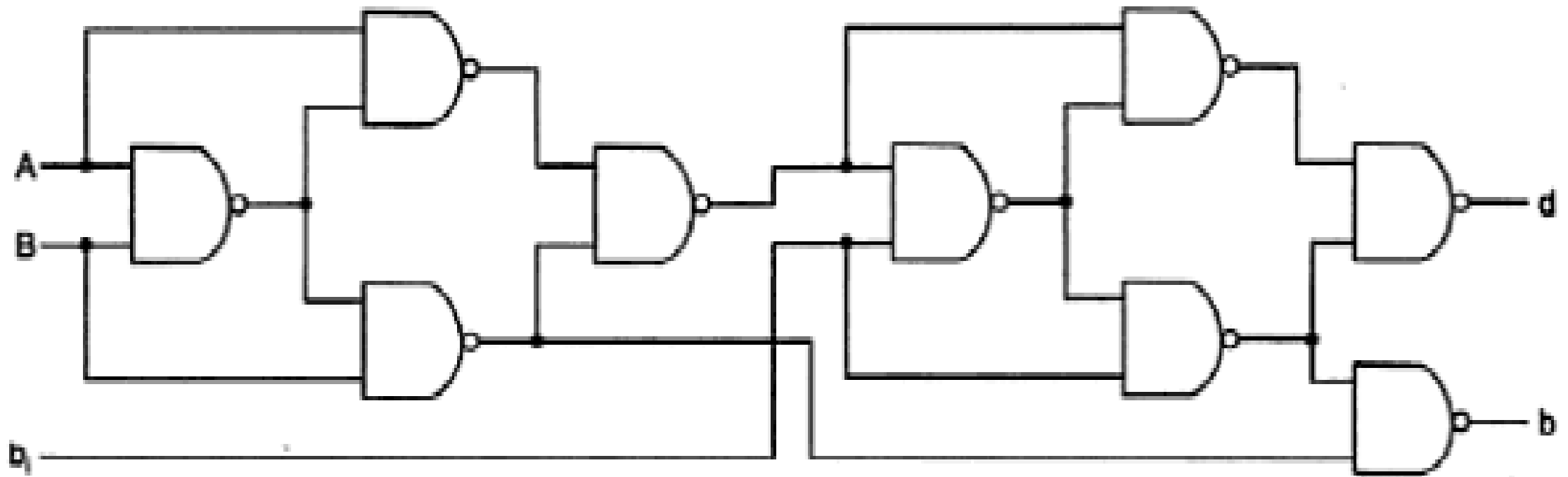
From the truth table, a circuit that will produce the correct difference and borrow bits in response to every possible combinations of A,B and b_i is

$$\begin{aligned}d &= \overline{A}\overline{B}b_i + \overline{A}B\overline{b}_i + A\overline{B}\overline{b}_i + ABb_i \\&= b_i(AB + \overline{A}\overline{B}) + \overline{b}_i(A\overline{B} + \overline{A}B) \\&= b_i(\overline{A \oplus B}) + \overline{b}_i(A \oplus B) = A \oplus B \oplus b_i\end{aligned}$$

and

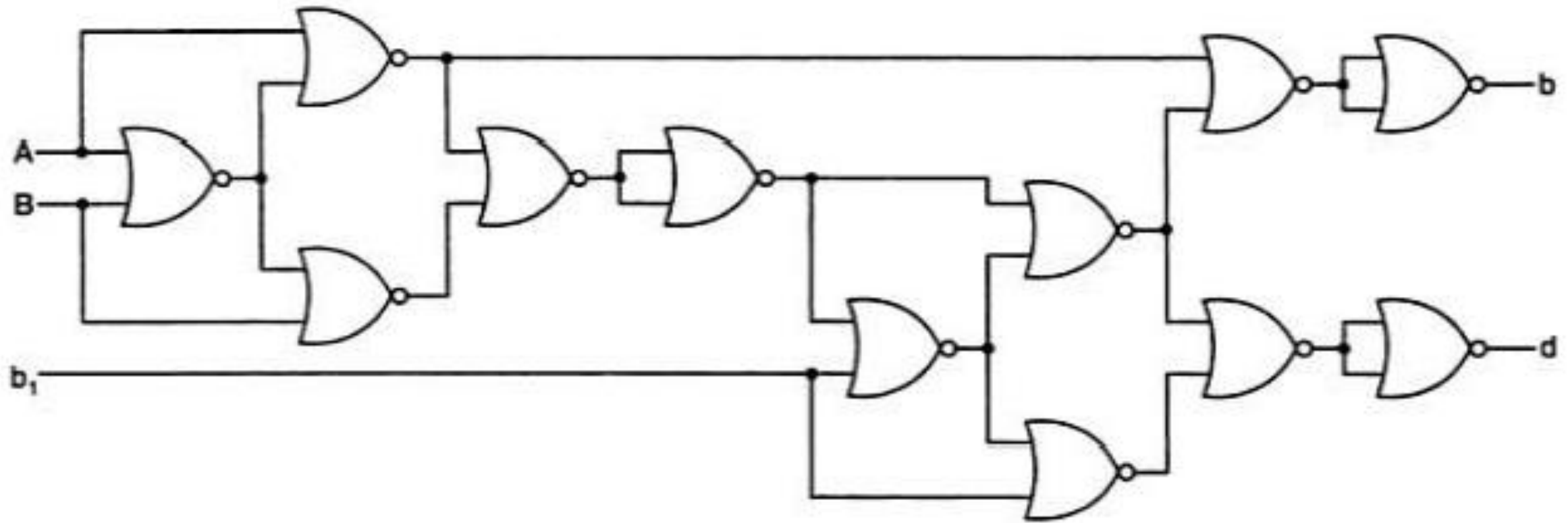
$$\begin{aligned}b &= \overline{A}\overline{B}b_i + \overline{A}B\overline{b}_i + \overline{A}Bb_i + ABb_i = \overline{A}B(b_i + \overline{b}_i) + (AB + \overline{A}\overline{B})b_i \\&= \overline{A}B + (\overline{A \oplus B})b_i\end{aligned}$$

NAND Logic for Full Subtractor



Logic diagram of a full-subtractor using only 2-input NAND gates.

NOR Logic for Full Subtractor



Logic diagram of a full subtractor using only 2-input NOR gates.

Parallel Adders

Types of Parallel Adders

Ripple Carry Adder (RCA)

Simple structure, but slow due to carry propagation delay.

Carry Look-Ahead Adder (CLA)

Faster by reducing carry propagation delay.

Carry Save Adder (CSA)

Used for high-speed multiplication operations.

Parallel Adders

Type	Speed	Structure Complexity	Application
Ripple Carry Adder	Slow	Simple	Small circuits
Carry Look-Ahead Adder	Fast	Moderate	ALUs, Processors
Carry Save Adder	Fastest	Complex	Multipliers

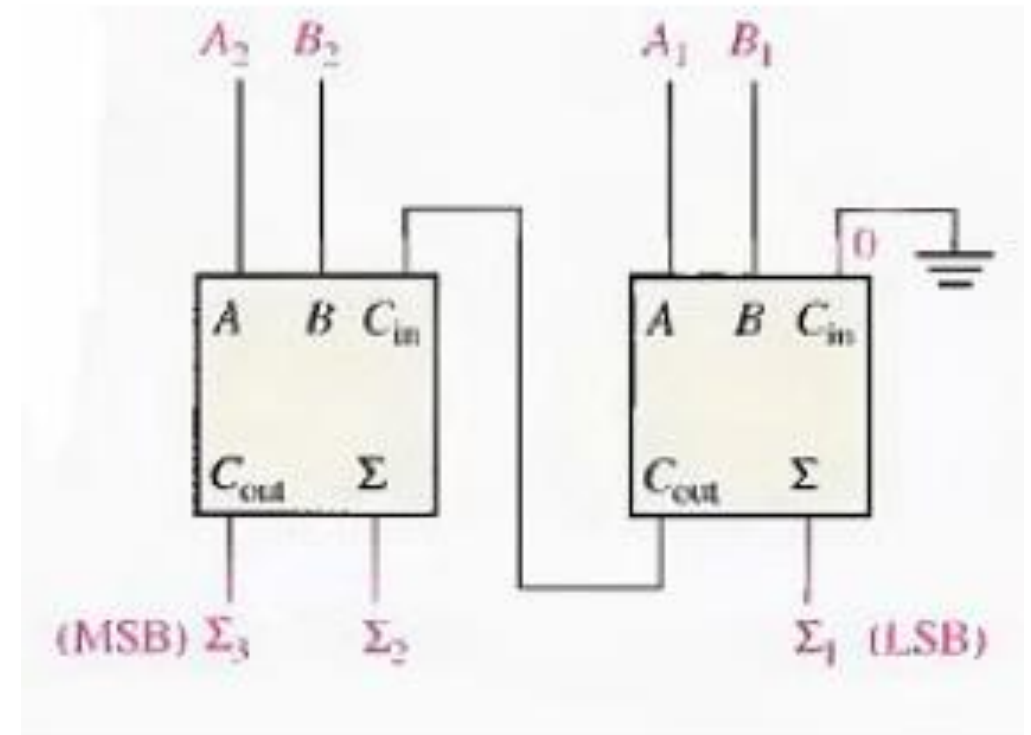
Applications of Parallel Adders

- Used in **Microprocessors & ALUs** for arithmetic operations.
- Essential for **Digital Signal Processing (DSP)** applications.
- Key component in **High-Speed Multipliers**.
- Utilized in **Floating Point Arithmetic** calculations.

Binary Parallel Adders

To add binary numbers with more than one bit, you must use additional full-adders. When one binary number is added to another, each column generates a sum bit and a 1 or 0 carry bit to the next column to the left, as illustrated here with 2-bit numbers.

To add two binary numbers, a full-adder is required for each bit in the numbers. So for 2-bit numbers, two adders are needed; for 4-bit numbers, four adders are used; and so on. The carry output of each adder is connected to the carry input of the next higher-order adder, as shown in Fig for two bits.

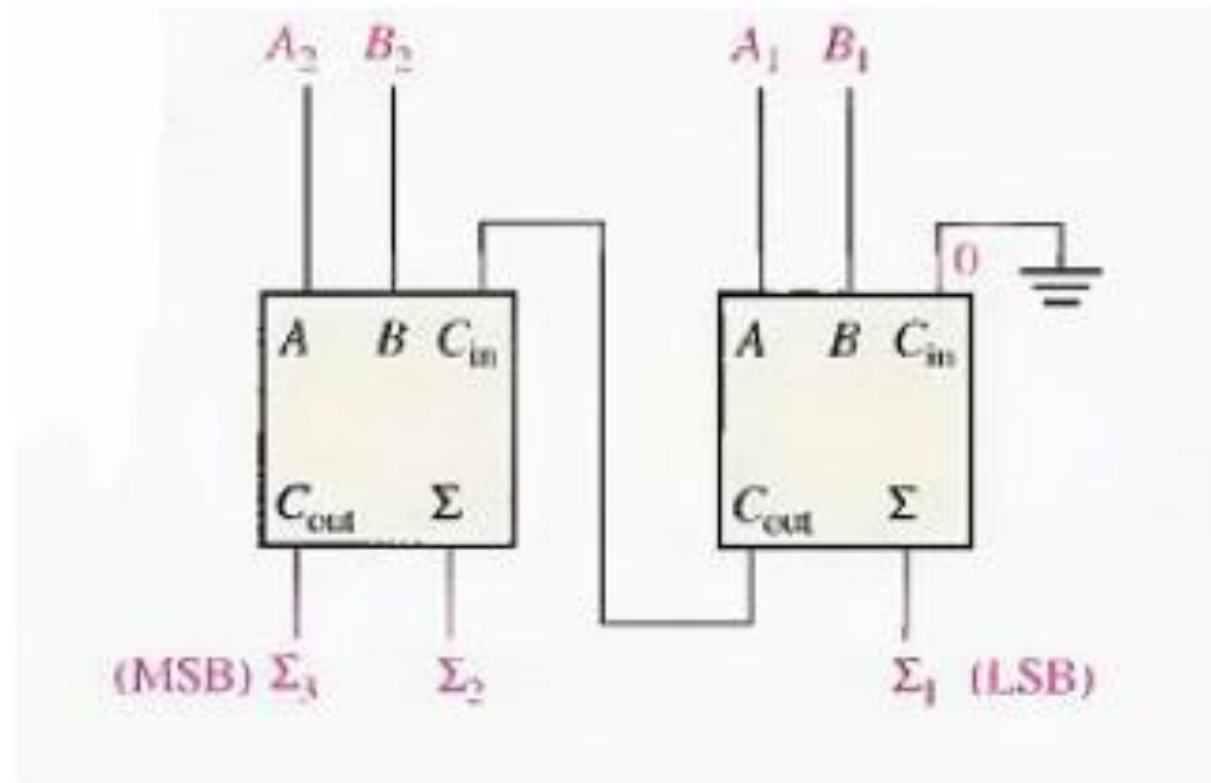


Parallel Adders

Ripple Carry Adder (RCA)

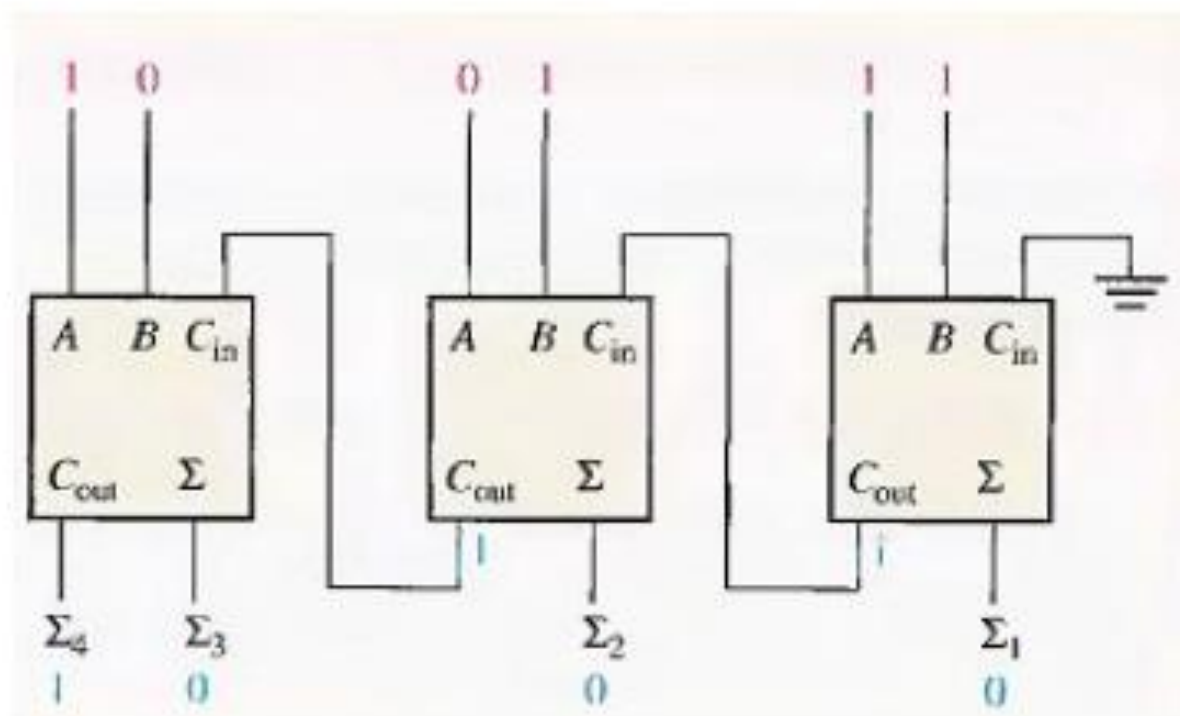
- Composed of multiple Full Adders connected in series.
- Each Full Adder adds two bits and a carry bit.
- The carry from each adder propagates to the next stage.
- Major drawback: Slow operation due to carry propagation.

2 Bit Parallel Adders



Block diagram of a basic 2-bit parallel adder using two full-adders.

Example: Determine the sum generated by the 3-bit parallel adder and show the intermediate carries when the binary numbers 101 and 011 are being added.



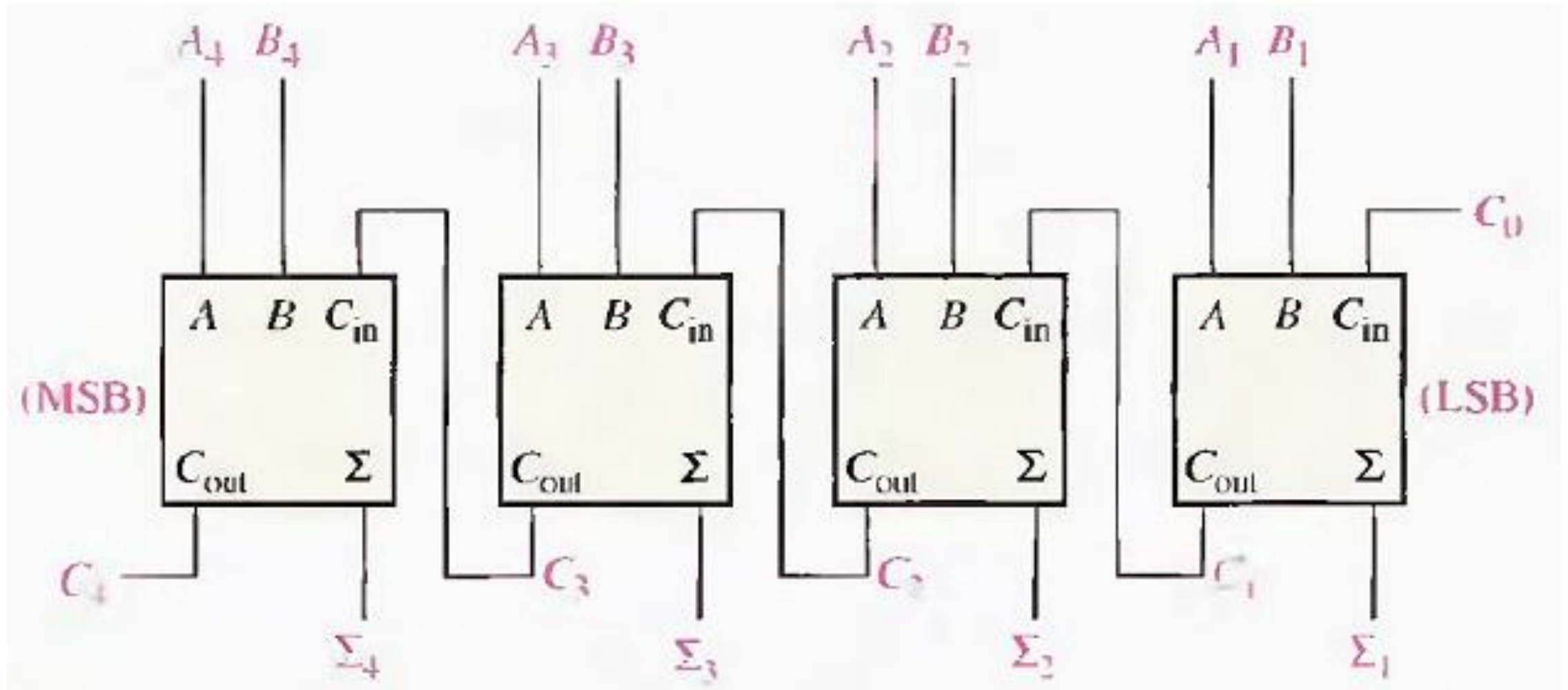
4 Bit Parallel Adders

A basic 4-bit parallel adder is implemented with four full-adder stages.

Again, the LSBs (A₁ and B₁) in each number being added go into the right-most full-adder: the higher-order bits are applied as shown to the successively higher-order adders, with the MSBs (A and B) in each number being applied to the left-most full-adder.

The Carry output of each adder is connected to the carry input of the next higher-order adder as indicated. These are called **internal carries**.

4 Bit Parallel Adders

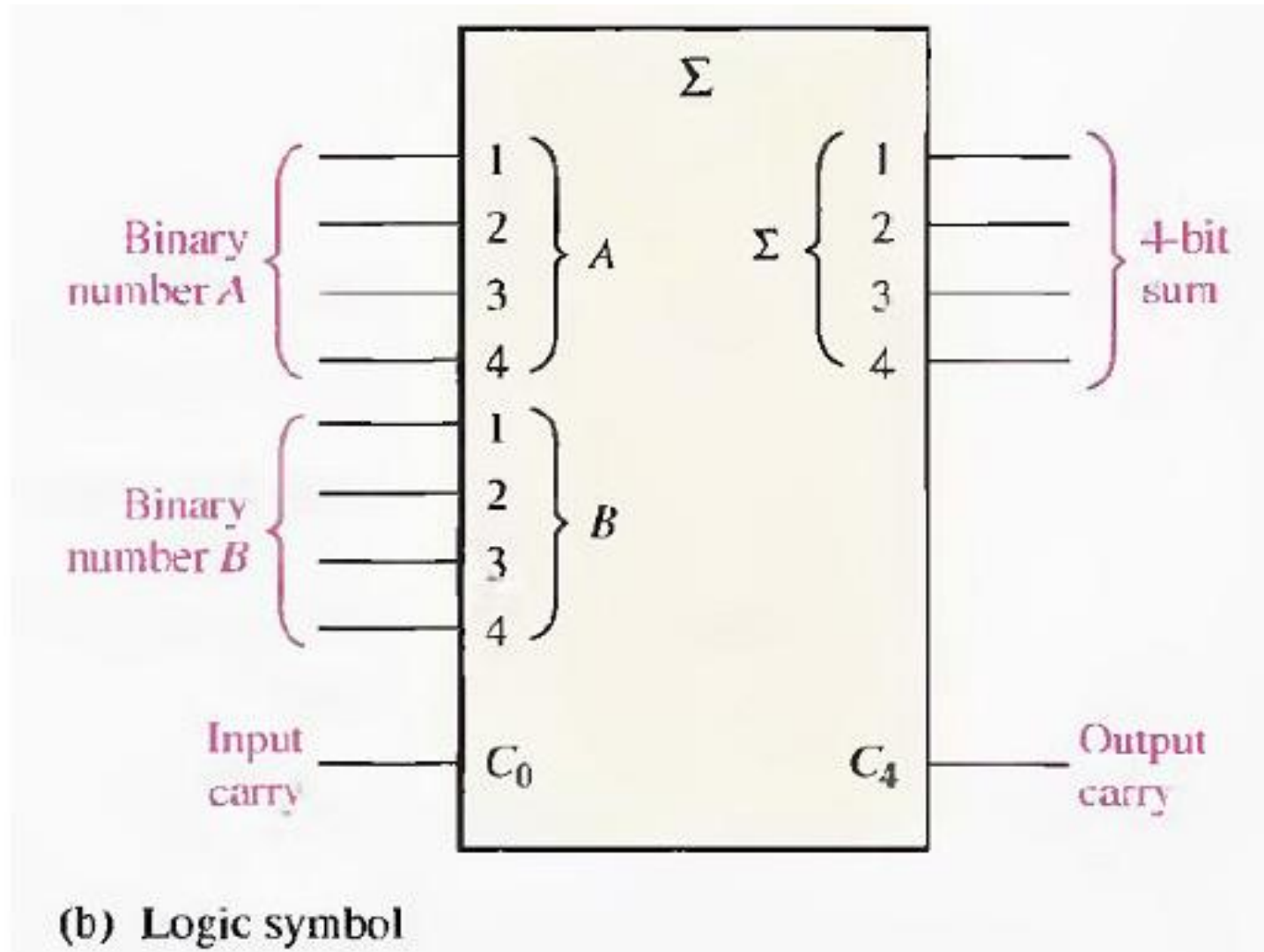


4 Bit Parallel Adders

Drawbacks of Ripple Parallel adder

- At each stage the sum bit is not valid unless the carry bits in all the preceding stages are valid.
- In effect, the carry bit must propagate or ripple, through all stages before the most significant sum bit is valid.
- These are called ripple carry, and this error is removed in carry look ahead adder.

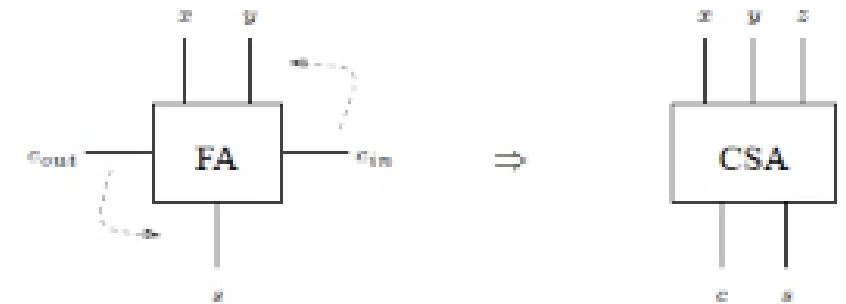
4 Bit Parallel Adders



Carry save Adders

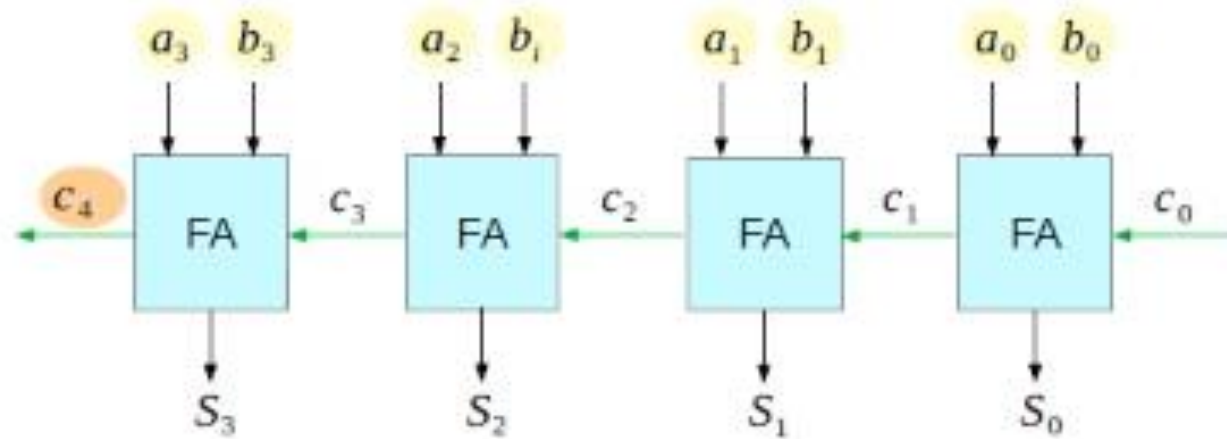
A carry save adder is typically used in a binary multiplier, since a binary multiplier involves addition of more than two binary numbers after multiplication.

- Used in fast multipliers.
- Instead of propagating carry, it stores it in a separate sum and carry register.
- Advantage: Faster than RCA for multiple additions.
- Disadvantage: Requires extra processing to combine results.

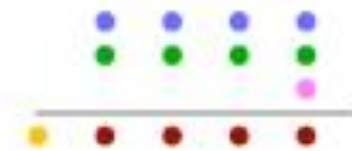


00011	A
00001	B
+ <u>01001</u>	C
01011	sum, excluding carries
+ <u>0001</u>	carries shifted left one place
01101	final sum

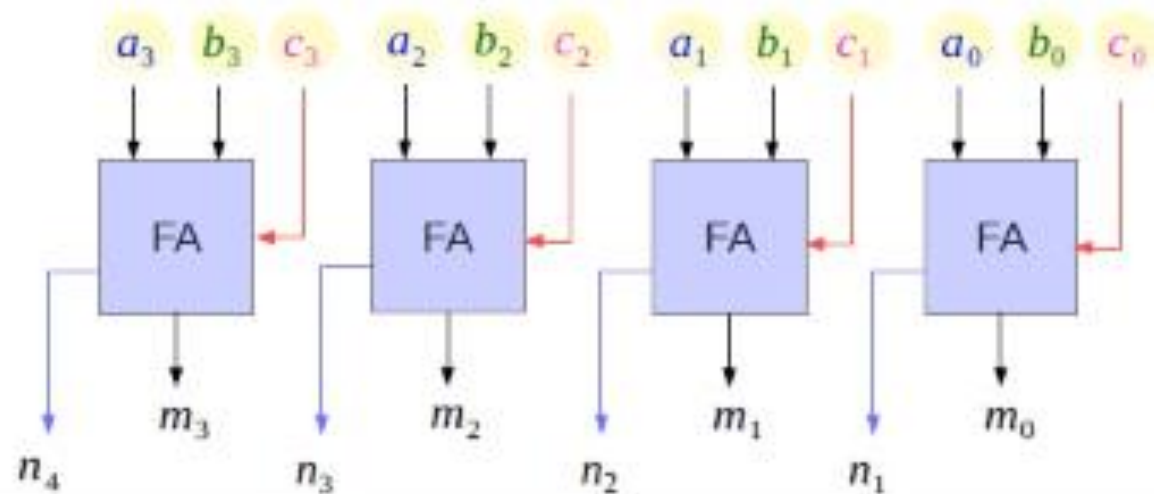
Carry save Adders



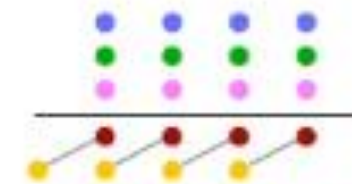
Ripple Carry Adder



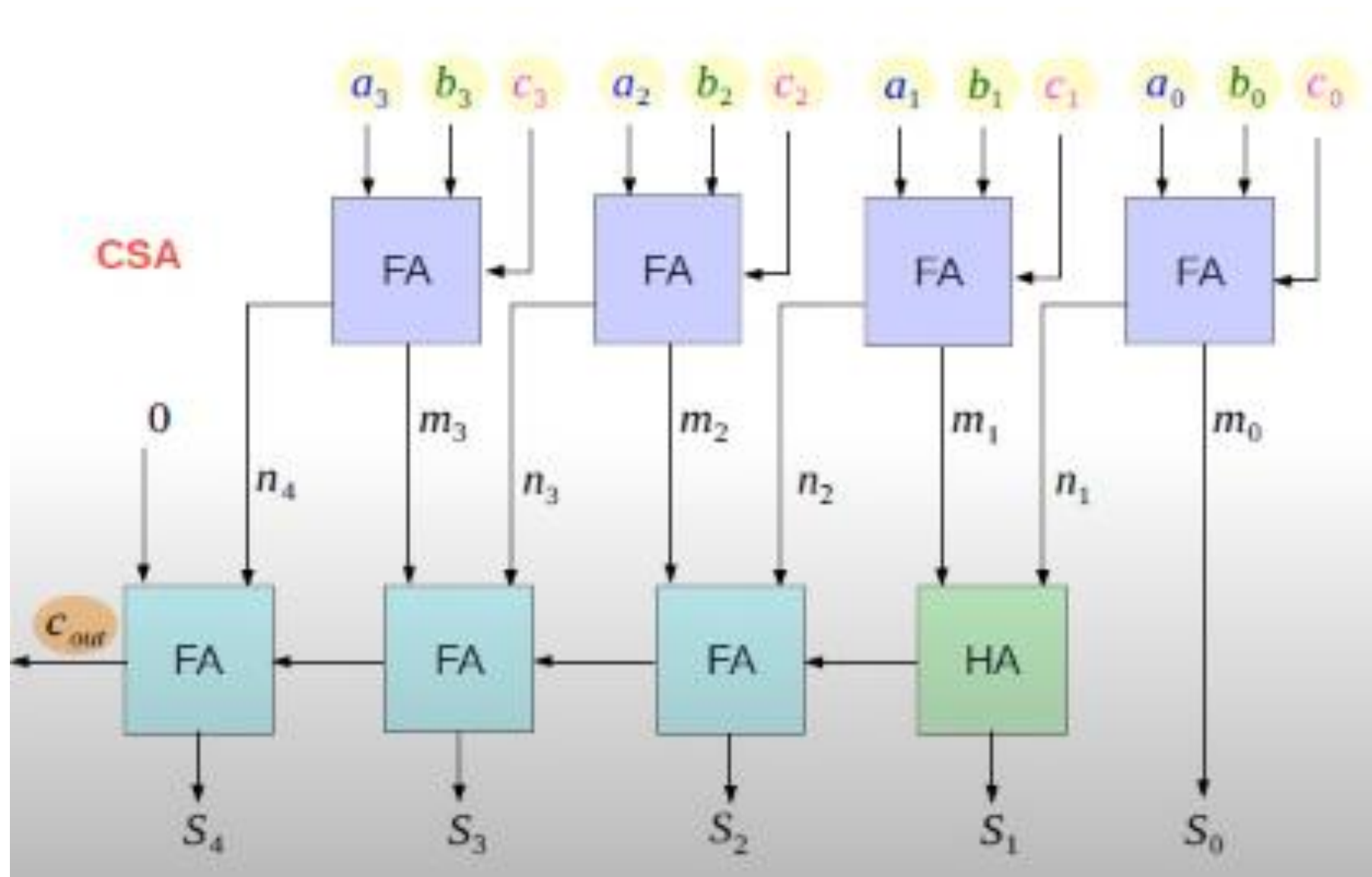
Carry Propagate Adder



Carry Save Adder



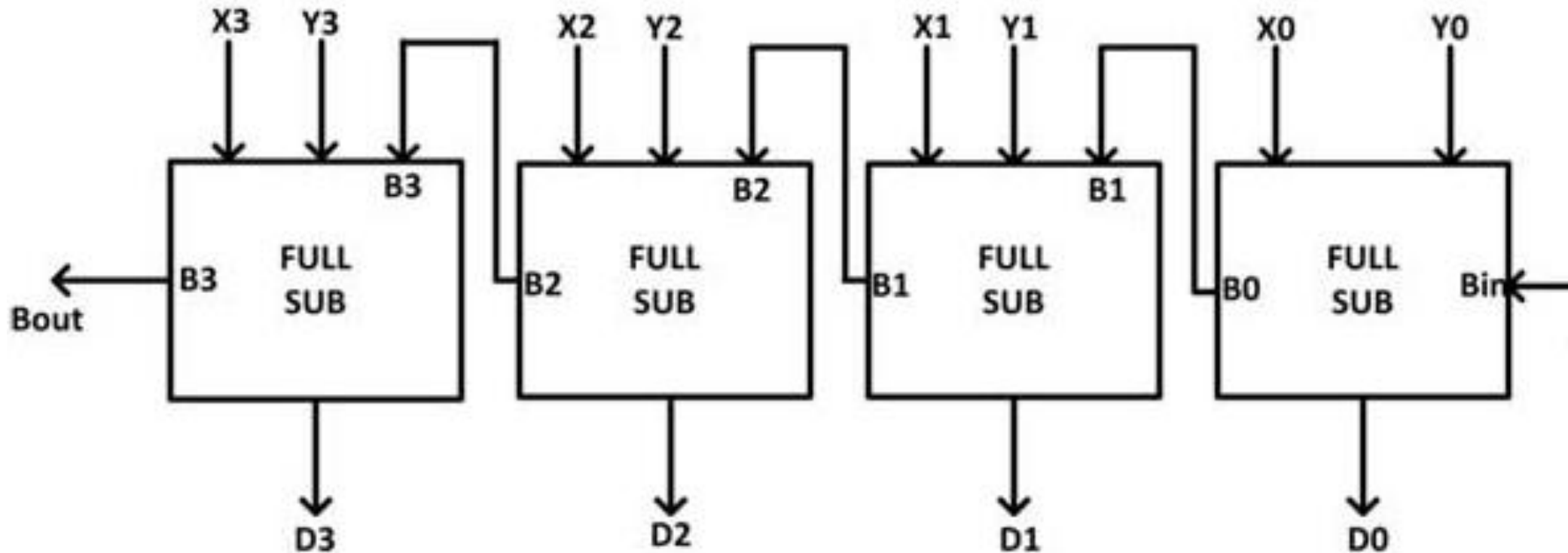
CSA and CPA



- As the design automatically avoids the delay in the carry-out bits, a CSA chain may be faster than using standard adders.
- In each level, outputs are produced in parallel, hence delay same as full adder
- Carry save adders are useful
 - when more than two operands are to be added
 - In the calculation of partial products in integer multiplication

2s Complement Adder/Subtractor

Consider the 4-bit subtractor



2s Complement Adder/Subtractor

We are going to design a single circuit which is capable to perform adder/subtractor functions simultaneously. That can be done using 2s complement adder/subtractor.

- This is done by including an X-OR gate with each full-adder.
- The mode input M controls the operation.

2s Complement Adder/Subtractor

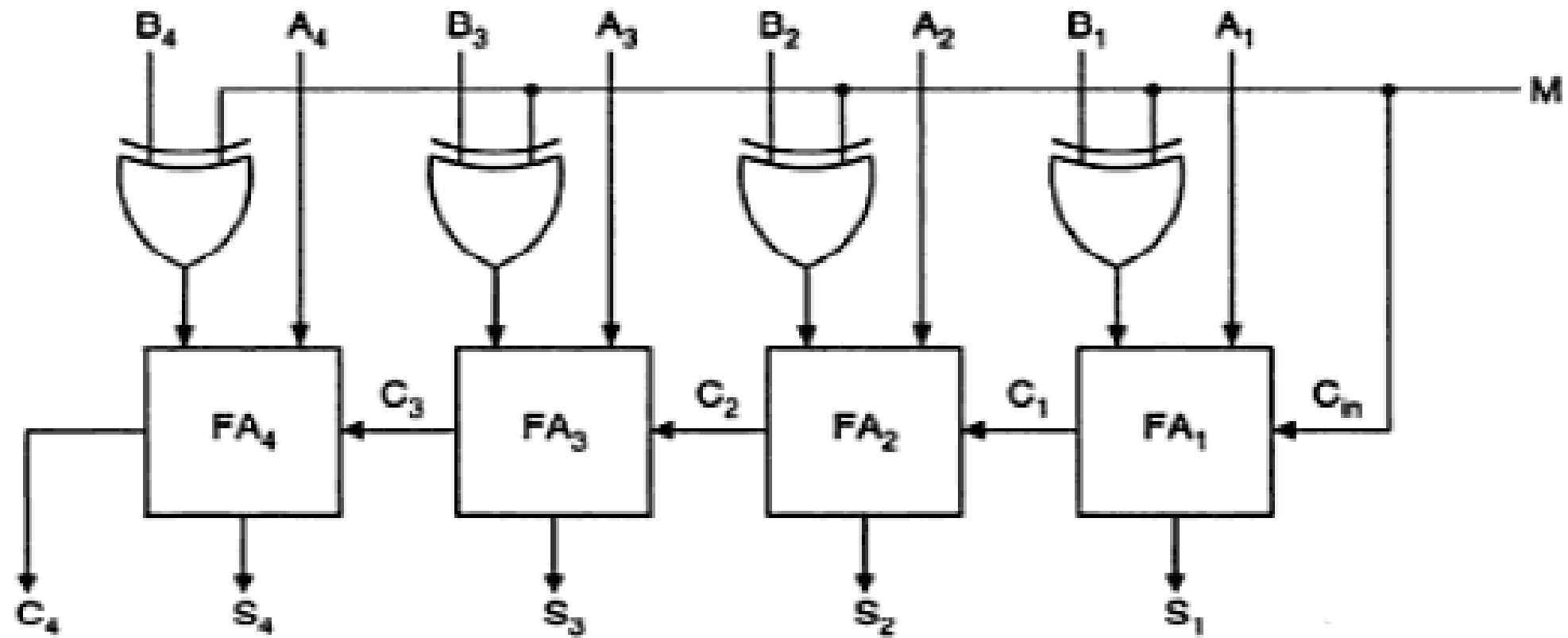
- When $M=0$, the circuit is an adder, and when $M=1$, the circuit becomes a subtractor.
- Each X-OR gate receives input M and one of the inputs of B .

When $M=0$, $B \text{ XOR } 0 = B$

- The full-adder receives the value of B , the input carry is 0
- and the circuit performs $A+B$.

When $B \text{ XOR } 1 = B'$ and $C1=1$.

The B inputs are complemented and a 1 is through the input carry. The circuit performs the operation A plus the 2's complement of B .



Logic diagram of a 4-bit binary adder-subtractor.

1s Complement Adder/Subtractor

- The 1's complement adder/subtractor is a method used in digital circuits to perform both addition and subtraction of binary numbers using 1's complement representation. It is widely used in arithmetic logic units (ALUs) and computing systems that support signed binary arithmetic.

1s Complement concept

- In 1's complement representation, negative numbers are represented by inverting all the bits of their positive counterpart.

For example, in an 8-bit system:

(+5) in binary: 00000101

(-5) in 1's complement: 11111010 (invert all bits of 00000101)

1s Complement adder/subtractor

Disadvantages

- The presence of negative zero ($111\dots111$) creates ambiguity.
- Requires handling of the end-around carry.

Applications

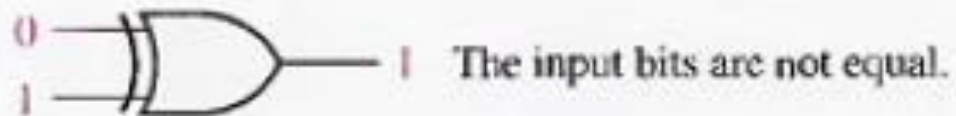
- Used in older computing systems and some modern ALUs.

Comparators

The basic function of a comparator is to compare the magnitudes of two binary quantities to determine the relationship of those quantities. In its simplest form, a comparator circuit determines whether two numbers are equal.

Equality

The exclusive-OR gate can be used as a basic comparator because its output is a 1 if the two input bits are not equal and a 0 if the input bits are equal. Fig.(7-11) shows the exclusive-OR gate as a 2-bit comparator.



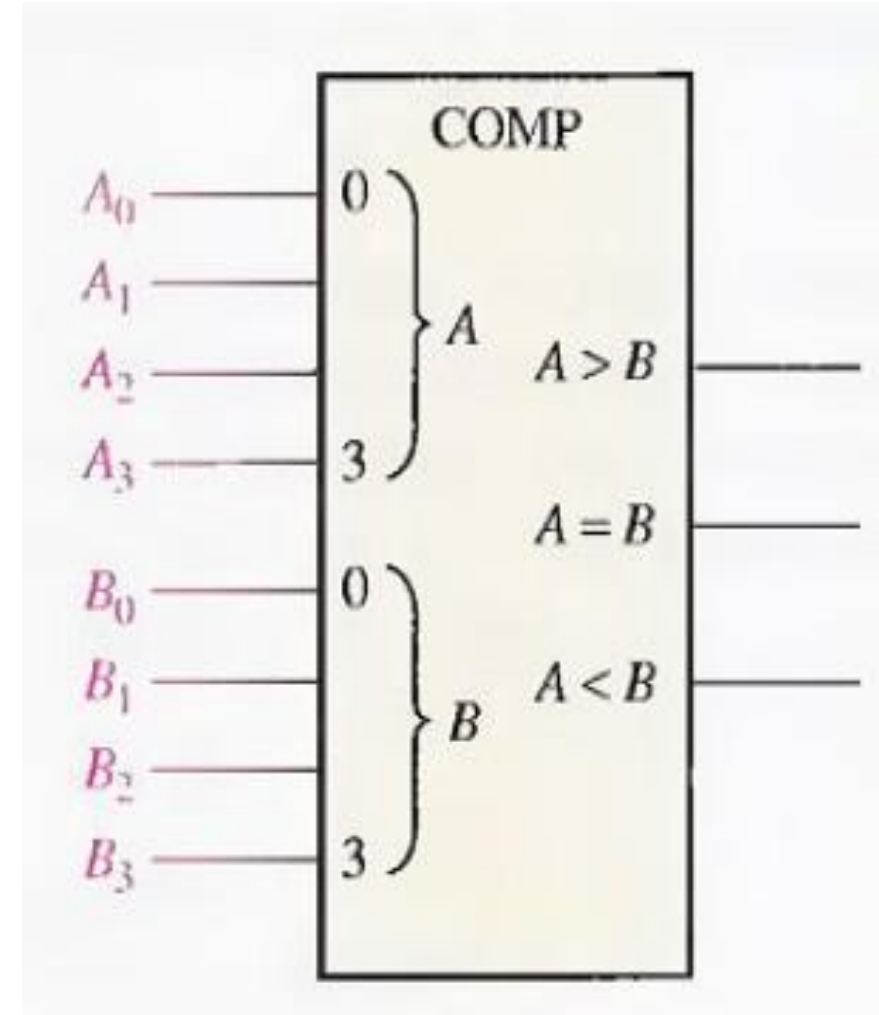
Comparators

Inequality

In addition to the equality output, many IC comparators provide additional outputs that indicate which of the two binary numbers being compared is the larger.

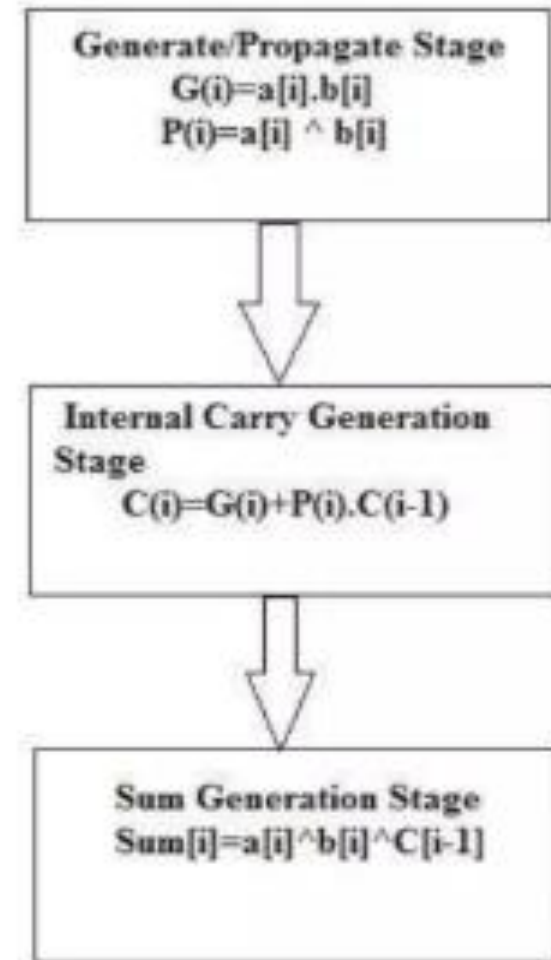
That is, there is an output that indicates when number A is greater than number B ($A > B$) and an output that indicates when number A is less than number B ($A < B$), as shown in the logic symbol for a 4-bit comparator.

To determine an inequality of binary numbers A and B, you first examine the highest-order bit in each number.



Carry Look-ahead Adder

Carry Look Ahead (CLA) design is based on the principle of looking at lower adder bits of argument and addend if higher orders carry generated. This adder reduces the carry delay by reducing the number of gates through which a carry signal must propagate. As shown in fig. in the generation and propagation stage, the generation values, propagation values are computed. Internal carry generation is calculated in second stage. And in final stage, the sum is calculated. The flow chart of CLA is given in fig. and the architecture of CLA is given in fig.



Flow chart of CLA

Thank you!